

Design Microservices Architecture with Java Spring Boot - Patterns & Best Practices

Table of Contents

1. [Introduction to Spring Boot Microservices](#)
 2. [Microservices Fundamentals](#)
 3. [Step-by-step Design Process](#)
 4. [Monolith to Microservices Evolution](#)
 5. [Spring Cloud Components](#)
 6. [Code Examples & Implementation](#)
 7. [Best Practices & Design Patterns](#)
 8. [Testing Strategies](#)
 9. [Deployment with Docker/Kubernetes](#)
 10. [Real-world Case Studies](#)
 11. [Security & Data Management](#)
 12. [Conclusion & Next Steps](#)
-

1. Introduction to Spring Boot Microservices {#introduction}

What are Microservices?

Microservices architecture is a method of developing software applications as a suite of independently deployable, small, modular services. Each service runs in its own process and communicates via well-defined, lightweight mechanisms, such as HTTP APIs.

Benefits of Microservices

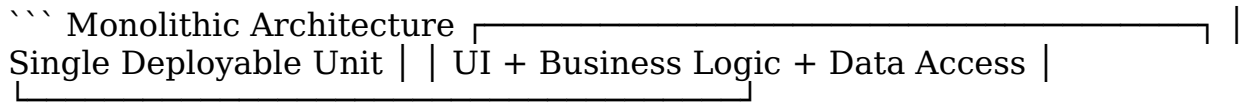
- **Scalability:** Scale individual services based on demand
- **Resilience:** Failure in one service doesn't bring down the entire system
- **Technology Diversity:** Use different technologies for different services
- **Team Autonomy:** Small teams can work independently
- **Faster Deployment:** Deploy services independently

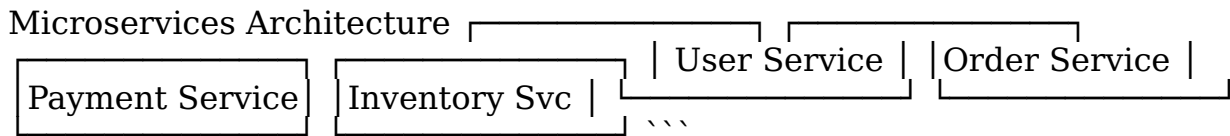
Challenges to Consider

- **Complexity:** Distributed systems are inherently complex
- **Data Consistency:** Managing transactions across services
- **Network Latency:** Inter-service communication overhead
- **Monitoring:** Observability across multiple services

- **Testing:** End-to-end testing complexity

Monolithic vs Microservices Architecture

Monolithic Architecture 

Microservices Architecture 

2. Spring Boot Microservices Fundamentals {#fundamentals}

Spring Boot Features

- **Convention over Configuration**
- **Embedded Web Servers**
- **Auto-configuration**
- **Production-ready Actuator**
- **Extensive Spring Ecosystem**

Spring Cloud Components

- **Spring Cloud Gateway**
- **Spring Cloud Config**
- **Spring Cloud Netflix**
- **Spring Cloud Circuit Breaker**
- **Spring Cloud Sleuth**

Core Principles

- **Domain-Driven Design**
- **Single Responsibility**
- **Loose Coupling**
- **High Cohesion**
- **Autonomous Services**

12-Factor App Methodology for Cloud-Native Applications

1. **Codebase:** One codebase tracked in revision control, many deploys
2. **Dependencies:** Explicitly declare and isolate dependencies
3. **Config:** Store config in the environment
4. **Backing services:** Treat backing services as attached resources
5. **Build, release, run:** Strictly separate build and run stages
6. **Processes:** Execute the app as one or more stateless processes
7. **Port binding:** Export services via port binding

8. **Concurrency:** Scale out via the process model
 9. **Disposability:** Maximize robustness with fast startup and graceful shutdown
 10. **Dev/prod parity:** Keep development, staging, and production as similar as possible
 11. **Logs:** Treat logs as event streams
 12. **Admin processes:** Run admin/management tasks as one-off processes
-

3. Step-by-Step Design Process {#design-process}

1. Domain Analysis

Identify business domains and bounded contexts using Domain-Driven Design principles

2. Service Decomposition

Break down domains into cohesive, loosely coupled services

3. API Design

Define service contracts and communication patterns

4. Data Strategy

Implement database per service pattern and data consistency strategies

5. Infrastructure

Set up service discovery, API gateway, and configuration management

6. Resilience

Implement circuit breakers, retries, and fault tolerance patterns

Design Decision Framework

When to Use Microservices:

- Independent team scaling
- Different technology requirements
- Independent scaling needs
- Enhanced security isolation
- Different change frequencies

When to Consider Modular Monolith:

- Small team size
 - Simple business domain
 - Limited operational expertise
 - Tight coupling between components
 - Consistent technology stack
-

4. Monolith to Microservices Evolution {#evolution}

Migration Strategies

Strangler Fig Pattern

Gradually replace monolithic functionality with microservices:

1. Identify service boundaries
2. Implement new functionality as microservices
3. Route traffic to new services
4. Gradually retire monolithic components

Database Decomposition

Split shared databases:

1. Analyze data dependencies
2. Extract service-specific data
3. Implement data synchronization
4. Handle eventual consistency

Migration Example: Extracting User Service

```
```java // 1. Original Monolithic User Controller @RestController public
class MonolithUserController { @Autowired private UserService
userService;

@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
 return userService.findById(id);
}

}

// 2. New Microservice User Controller @RestController
@RequestMapping("/api/v1/users") public class UserController
{ @Autowired private UserService userService;
```

```

@GetMapping("/{id}")
public ResponseEntity<UserDTO> getUser(@PathVariable Long id) {
 UserDTO user = userService.findById(id);
 return ResponseEntity.ok(user);
}

}

// 3. API Gateway Routing Configuration @Configuration @EnableZuulProxy
public class GatewayConfig {

@Bean
public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
 return builder.routes()
 .route("user-service", r -> r.path("/users/**")
 .uri("lb://user-service"))
 .build();
}

} ``

```

---

## 5. Spring Cloud Components Implementation {#spring-cloud}

### Spring Cloud Gateway

**Purpose:** API Gateway for routing, filtering, and cross-cutting concerns

- Request routing and load balancing
- Authentication and authorization
- Rate limiting and circuit breaking
- Request/response transformation

### Spring Cloud Config

**Purpose:** Centralized configuration management

- External configuration storage
- Environment-specific configs
- Dynamic configuration updates
- Configuration versioning

### Service Discovery (Eureka)

**Purpose:** Service registration and discovery

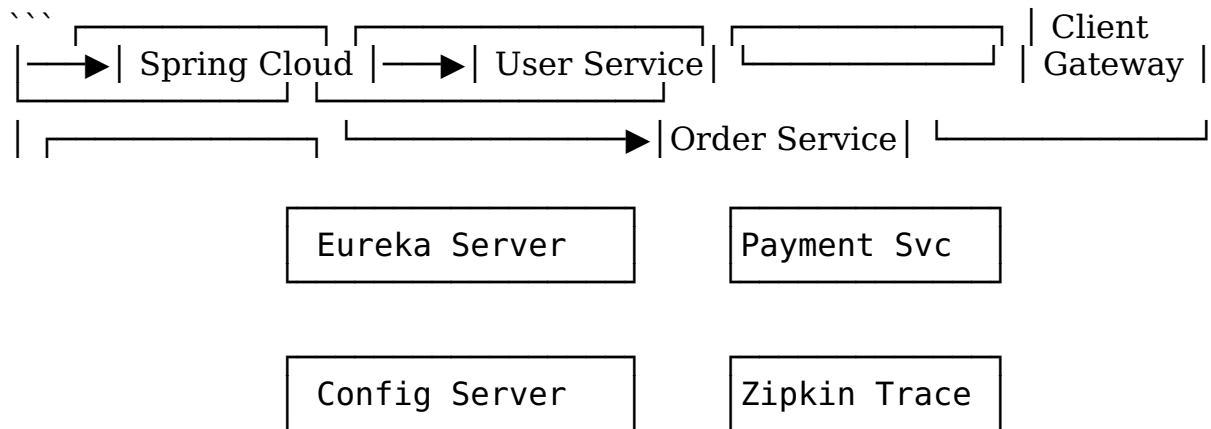
- Automatic service registration
- Health checking
- Load balancing integration
- Failover support

## Circuit Breaker (Resilience4j)

**Purpose:** Fault tolerance and resilience

- Prevent cascading failures
- Fallback mechanisms
- Retry policies
- Bulkhead isolation

# Spring Cloud Ecosystem Architecture



## 6. Code Examples & Practical Implementation

## 1. Spring Cloud Config Server Setup

```
java // ConfigServerApplication.java @SpringBootApplication
@EnableConfigServer @EnableEurekaClient public class
ConfigServerApplication { public static void main(String[] args)
{ SpringApplication.run(ConfigServerApplication.class, args); } }
```

```
```yaml
```

application.yml

```
server: port: 8888
```

```
spring: application: name: config-server cloud: config: server: git: uri:
https://github.com/your-org/config-repo search-paths: '{application}'
default-label: main native: search-locations: classpath:/configurations/
profiles: active: native
```

```
eureka: client: service-url: defaultZone: http://localhost:8761/eureka
register-with-eureka: true fetch-registry: true ``
```

2. Eureka Service Discovery Setup

```
java // EurekaServerApplication.java @SpringBootApplication
@EnableEurekaServer public class EurekaServerApplication { public
static void main(String[] args)
{ SpringApplication.run(EurekaServerApplication.class, args); } }
```

```yaml

## application.yml

```
server: port: 8761
```

```
spring: application: name: eureka-server
```

```
eureka: instance: hostname: localhost client: register-with-eureka: false
fetch-registry: false service-url: defaultZone: http://$
{eureka.instance.hostname}:${server.port}/eureka/ ```
```

## 3. Spring Cloud Gateway Configuration

```
```java // GatewayApplication.java @SpringBootApplication
@EnableEurekaClient public class GatewayApplication {
```

```
public static void main(String[] args) {
    SpringApplication.run(GatewayApplication.class, args);
}
```

```
@Bean
```

```
public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
    return builder.routes()
        .route("user-service", r -> r.path("/api/users/**")
            .filters(f -> f.stripPrefix(1))
            .uri("lb://user-service"))
        .route("order-service", r -> r.path("/api/orders/**")
            .filters(f -> f.stripPrefix(1)
                .circuitBreaker(config -> config
                    .setName("orderCircuitBreaker")
                    .setFallbackUri("forward:/fallback/orders")))
            .uri("lb://order-service"))
        .route("payment-service", r -> r.path("/api/payments/**")
            .filters(f -> f.stripPrefix(1)
                .requestRateLimiter(config -> config
                    .setRateLimiter(redisRateLimiter())
                    .setKeyResolver(userKeyResolver())))
            .uri("lb://payment-service"))
        .build();
}
```

```
@Bean
```

```
public RedisRateLimiter redisRateLimiter() {
```

```

        return new RedisRateLimiter(10, 20, 1);
    }

    @Bean
    public KeyResolver userKeyResolver() {
        return exchange -> exchange.getRequest().getHeaders()
            .getFirst("user-id");
    }
}
```

```

## 4. Circuit Breaker with Resilience4j

```

```java // OrderService.java @Service @Slf4j public class OrderService {

    @Autowired
    private PaymentServiceClient paymentClient;

    @Autowired
    private InventoryServiceClient inventoryClient;

    @CircuitBreaker(name = "payment-service", fallbackMethod = "fallbackPayment")
    @Retry(name = "payment-service")
    @TimeLimiter(name = "payment-service")
    public CompletableFuture<PaymentResponse> processPayment(PaymentRequest request) {
        return CompletableFuture.supplyAsync(() -> {
            log.info("Processing payment for order: {}", request.getOrderId());
            return paymentClient.processPayment(request);
        });
    }

    @Bulkhead(name = "inventory-service", type = Bulkhead.Type.THREADPOOL)
    public InventoryResponse checkInventory(String productId, int quantity) {
        log.info("Checking inventory for product: {}", productId);
        return inventoryClient.checkAvailability(productId, quantity);
    }

    public CompletableFuture<PaymentResponse> fallbackPayment(PaymentRequest request, Throwable ex) {
        log.error("Payment service is down, using fallback: {}", ex.getMessage());
        return CompletableFuture.completedFuture(
            PaymentResponse.builder()
                .status("FAILED")
                .message("Payment service temporarily unavailable")
                .build()
        );
    }
}
```
```yaml
```

```

# resilience4j configuration

```
resilience4j: circuitbreaker: instances: payment-service: sliding-window-size:
10 failure-rate-threshold: 50 wait-duration-in-open-state: 30s permitted-
number-of-calls-in-half-open-state: 3 minimum-number-of-calls: 5 retry:
instances: payment-service: max-attempts: 3 wait-duration: 2s exponential-
backoff-multiplier: 2 bulkhead: instances: inventory-service: max-concurrent-
calls: 10 max-wait-duration: 1s timelimiter: instances: payment-service:
timeout-duration: 5s ```
```

## 5. Complete Microservice Example - User Service

### User Entity & Repository

```
```java @Entity @Table(name = "users") @Data @NoArgsConstructor
@AllArgsConstructor @Builder public class User { @Id
@GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;

@Column(unique = true, nullable = false)
@email(message = "Email should be valid")
private String email;

@NotBlank(message = "First name is required")
private String firstName;

@NotBlank(message = "Last name is required")
private String lastName;

@Embedded
private Address address;

@CreationTimestamp
private LocalDateTime createdAt;

@UpdateTimestamp
private LocalDateTime updatedAt;

}

@Embeddable @Data public class Address { private String street; private
String city; private String state; private String zipCode; private String
country; }

@Repository public interface UserRepository extends JpaRepository
{ Optional findByEmail(String email); List findByAddressCity(String city);

@Query("SELECT u FROM User u WHERE u.firstName LIKE %?1% OR u.lastName LIK
List<User> findByNameContaining(String name);

} ```
```

User Service & Controller

```
``java @Service @Slf4j @Transactional public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private UserMapper userMapper;

    public UserDto createUser(CreateUserRequest request) {
        log.info("Creating new user with email: {}", request.getEmail());

        if (userRepository.findByEmail(request.getEmail()).isPresent()) {
            throw new UserAlreadyExistsException("User with email " + request.
        }

        User user = User.builder()
            .email(request.getEmail())
            .firstName(request.getFirstName())
            .lastName(request.getLastName())
            .address(request.getAddress())
            .build();

        User savedUser = userRepository.save(user);
        log.info("User created successfully with ID: {}", savedUser.getId());

        return userMapper.toDto(savedUser);
    }

    @Transactional(readOnly = true)
    public UserDto getUserById(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException("User not found with
        return userMapper.toDto(user);
    }

    @Transactional(readOnly = true)
    public List<UserDto> getAllUsers() {
        return userRepository.findAll().stream()
            .map(userMapper::toDto)
            .collect(Collectors.toList());
    }
}

@RestController @RequestMapping("/api/v1/users") @Validated @Slf4j
public class UserController {

    @Autowired
    private UserService userService;
```

```
@PostMapping
public ResponseEntity<UserDto> createUser(@Valid @RequestBody CreateUserRe
    UserDto user = userService.createUser(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(user);
}

@GetMapping("/{id}")
public ResponseEntity<UserDto> getUser(@PathVariable Long id) {
    UserDto user = userService.getUserById(id);
    return ResponseEntity.ok(user);
}

@GetMapping
public ResponseEntity<List<UserDto>> getAllUsers() {
    List<UserDto> users = userService.getAllUsers();
    return ResponseEntity.ok(users);
}

} ````
```

7. Best Practices & Design Patterns {#best-practices}

Data Management Patterns

- **Database per Service:** Each microservice owns its data
- **Saga Pattern:** Manage distributed transactions
- **CQRS:** Separate read and write operations
- **Event Sourcing:** Store events instead of current state

Communication Patterns

- **API Gateway:** Single entry point for clients
- **Service Mesh:** Infrastructure layer for service communication
- **Event-Driven:** Asynchronous messaging between services
- **Request-Response:** Synchronous communication patterns

Resilience Patterns

- **Circuit Breaker:** Prevent cascading failures
- **Bulkhead:** Isolate critical resources
- **Timeout:** Prevent hanging requests
- **Retry:** Handle transient failures

Observability Patterns

- **Distributed Tracing:** Track requests across services
- **Metrics Collection:** Monitor system health
- **Centralized Logging:** Aggregate logs from all services

- **Health Checks:** Monitor service availability

Saga Pattern Implementation Example

```
```java @Component @Slf4j public class OrderSagaOrchestrator {

@Autowired
private PaymentServiceClient paymentClient;

@Autowired
private InventoryServiceClient inventoryClient;

@Autowired
private OrderRepository orderRepository;

@EventListener
public void handleOrderCreated(OrderCreatedEvent event) {
 log.info("Starting order saga for order: {}", event.getOrderId());

 try {
 // Step 1: Reserve inventory
 InventoryReservationRequest reservationRequest = InventoryReservationRequest.builder()
 .orderId(event.getOrderId())
 .items(event.getOrderItems())
 .build();

 inventoryClient.reserveItems(reservationRequest);

 } catch (Exception e) {
 log.error("Inventory reservation failed for order: {}", event.getOrderId());
 handleSagaFailure(event.getOrderId(), "INVENTORY_RESERVATION_FAILED");
 }
}

@EventListener
public void handleInventoryReserved(InventoryReservedEvent event) {
 log.info("Inventory reserved for order: {}", event.getOrderId());

 try {
 // Step 2: Process payment
 PaymentRequest paymentRequest = PaymentRequest.builder()
 .orderId(event.getOrderId())
 .amount(event.getTotalAmount())
 .customerId(event.getCustomerId())
 .build();

 paymentClient.processPayment(paymentRequest);

 } catch (Exception e) {
 log.error("Payment processing failed for order: {}", event.getOrderId());
 // Compensate: Release inventory
 inventoryClient.releaseReservation(event.getOrderId());
 }
}
}
```

```

 handleSagaFailure(event.getOrderId(), "PAYMENT_FAILED");
 }
}

@EventListener
public void handlePaymentProcessed(PaymentProcessedEvent event) {
 log.info("Payment processed for order: {}", event.getOrderId());

 // Step 3: Confirm order
 Order order = orderRepository.findById(event.getOrderId())
 .orElseThrow(() -> new OrderNotFoundException("Order not found"));

 order.setStatus(OrderStatus.CONFIRMED);
 orderRepository.save(order);

 // Publish order confirmed event
 eventPublisher.publishEvent(new OrderConfirmedEvent(event.getOrderId()))
}

private void handleSagaFailure(Long orderId, String reason) {
 Order order = orderRepository.findById(orderId)
 .orElseThrow(() -> new OrderNotFoundException("Order not found"));

 order.setStatus(OrderStatus.FAILED);
 order.setFailureReason(reason);
 orderRepository.save(order);

 eventPublisher.publishEvent(new OrderFailedEvent(orderId, reason));
}
} ``

```

---

## 8. Spring Boot Microservices Testing Strategies {#testing}

### Testing Layers

#### Unit Testing

- Test individual components in isolation
- Mock external dependencies
- Fast execution and feedback
- JUnit 5 + Mockito

#### Integration Testing

- Test service interactions
- Database integration tests
- Message broker testing

- Spring Boot Test slices

## End-to-End Testing

- Test complete user workflows
- Test across multiple services
- Production-like environment
- TestContainers integration

## Testing Implementation Examples

### Unit Test Example

```
```java @ExtendWith(MockitoExtension.class) class UserServiceTest {

@Mock
private UserRepository userRepository;

@Mock
private UserMapper userMapper;

@InjectMocks
private UserService userService;

@Test
@DisplayName("Should create user successfully")
void shouldCreateUserSuccessfully() {
    // Given
    CreateUserRequest request = CreateUserRequest.builder()
        .email("test@example.com")
        .firstName("John")
        .lastName("Doe")
        .build();

    User savedUser = User.builder()
        .id(1L)
        .email("test@example.com")
        .firstName("John")
        .lastName("Doe")
        .build();

    UserDto expectedDto = UserDto.builder()
        .id(1L)
        .email("test@example.com")
        .firstName("John")
        .lastName("Doe")
        .build();

    when(userRepository.findByEmail(request.getEmail())).thenReturn(Optional.of(savedUser));
    when(userRepository.save(any(User.class))).thenReturn(savedUser);
    when(userMapper.toDto(savedUser)).thenReturn(expectedDto);
}
```

```

    // When
    UserDto result = userService.createUser(request);

    // Then
    assertThat(result).isNotNull();
    assertThat(result.getId()).isEqualTo(1L);
    assertThat(result.getEmail()).isEqualTo("test@example.com");

    verify(userRepository).findByEmail(request.getEmail());
    verify(userRepository).save(any(User.class));
    verify(userMapper).toDto(savedUser);
}

@Test
@DisplayName("Should throw exception when user already exists")
void shouldThrowExceptionWhenUserAlreadyExists() {
    // Given
    CreateUserRequest request = CreateUserRequest.builder()
        .email("existing@example.com")
        .build();

    when(userRepository.findByEmail(request.getEmail()))
        .thenReturn(Optional.of(new User()));

    // When & Then
    assertThatThrownBy(() -> userService.createUser(request))
        .isInstanceOf(UserAlreadyExistsException.class)
        .hasMessage("User with email existing@example.com already exists");
}
}
}

```

Integration Test with TestContainers

```

```java @SpringBootTest @Testcontainers
@AutoConfigureTestDatabase(replace =
AutoConfigureTestDatabase.Replace.NONE) class
UserServiceIntegrationTest {

 @Container
 static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgr
 .withDatabaseName("testdb")
 .withUsername("test")
 .withPassword("test");

 @Autowired
 private UserService userService;

 @Autowired
 private TestRestTemplate restTemplate;
}

```

```

@dynamicPropertySource
static void configureProperties(DynamicPropertyRegistry registry) {
 registry.add("spring.datasource.url", postgres::getJdbcUrl);
 registry.add("spring.datasource.username", postgres::getUsername);
 registry.add("spring.datasource.password", postgres::getPassword);
}

@Test
@DisplayName("Should create and retrieve user")
void shouldCreateAndRetrieveUser() {
 // Given
 CreateUserRequest request = CreateUserRequest.builder()
 .email("integration@test.com")
 .firstName("Integration")
 .lastName("Test")
 .build();

 // When - Create user
 ResponseEntity<UserDto> createResponse = restTemplate.postForEntity(
 "/api/v1/users", request, UserDto.class);

 // Then - Verify creation
 assertThat(createResponse.getStatusCode()).isEqualTo(HttpStatus.CREATED);
 UserDto createdUser = createResponse.getBody();
 assertThat(createdUser).isNotNull();
 assertThat(createdUser.getEmail()).isEqualTo("integration@test.com");

 // When - Retrieve user
 ResponseEntity<UserDto> getResponse = restTemplate.getForEntity(
 "/api/v1/users/" + createdUser.getId(), UserDto.class);

 // Then - Verify retrieval
 assertThat(getResponse.getStatusCode()).isEqualTo(HttpStatus.OK);
 UserDto retrievedUser = getResponse.getBody();
 assertThat(retrievedUser).isEqualTo(createdUser);
}
}

```

## Contract Testing with Spring Cloud Contract

```

groovy // Contract Definition (Groovy DSL) Contract.make
{ description "should return user by id" request { method GET()
url "/api/v1/users/1" headers
{ contentType(applicationJson()) } } response { status OK()
headers { contentType(applicationJson()) } body([id: 1, email:
"test@example.com", firstName: "John", lastName: "Doe"]) } }

```java // Consumer Test @SpringBootTest @AutoConfigureStubRunner(ids
= ["com.example:user-service:+:stubs:8080"]) class
OrderServiceContractTest {

```

```
@Autowired
private OrderService orderService;

@Test
void shouldRetrieveUserFromContract() {
    // Given
    Long userId = 1L;

    // When
    UserDto user = orderService.getUserById(userId);

    // Then
    assertNotNull(user);
    assertEquals(1L, user.getId());
    assertEquals("test@example.com", user.getEmail());
}
} ````
```

9. Deployment with Docker & Kubernetes {#deployment}

Docker Containerization

Benefits:

- Consistent deployment environments
- Isolation and resource management
- Easy scaling and orchestration
- Version control for infrastructure

Kubernetes Orchestration

Features:

- Automated deployment and scaling
- Service discovery and load balancing
- Self-healing and rolling updates
- Configuration and secret management

Deployment Configuration Examples

Dockerfile for Spring Boot Microservice

```
````dockerfile
```

# Multi-stage build for optimized image size

FROM openjdk:17-jdk-slim as builder

WORKDIR /workspace/app COPY mvnw . COPY .mvn .mvn COPY pom.xml .  
COPY src src

## Download dependencies and build application

RUN ./mvnw install -DskipTests

## Extract layers for better caching

RUN mkdir -p target/dependency && (cd target/dependency; jar -xf ../\*.jar)

## Runtime stage

FROM openjdk:17-jre-slim

## Create non-root user for security

RUN addgroup --system spring && adduser --system spring --ingroup spring  
USER spring:spring

## Copy application layers

WORKDIR /app COPY --from=builder workspace/app/target/dependency/  
BOOT-INF/lib /app/lib COPY --from=builder workspace/app/target/  
dependency/META-INF /app/META-INF COPY --from=builder workspace/  
app/target/dependency/BOOT-INF/classes /app

## Health check

HEALTHCHECK --interval=30s --timeout=3s --start-period=60s --retries=3 \  
CMD curl -f http://localhost:8080/actuator/health || exit 1

EXPOSE 8080

```
ENTRYPOINT ["java", "-cp", "app:app/lib/*",
"com.example.userservice.UserServiceApplication"] ``
```

## **docker-compose.yml for Local Development**

```
``yaml version: '3.8'
```

```
services: # Infrastructure Services eureka-server: build: ./eureka-server
ports: - "8761:8761" environment: - SPRINGPROFILEACTIVE=docker
networks: - microservices-network
```

```
config-server: build: ./config-server ports: - "8888:8888" dependson: -
eureka-server environment: - SPRINGPROFILEACTIVE=docker -
EUREKACLIENTSERVICEURL_DEFAULTZONE=http://eureka-server:8761/
eureka networks: - microservices-network
```

```
api-gateway: build: ./api-gateway ports: - "8080:8080" dependson: - eureka-
server - config-server environment: - SPRINGPROFILEACTIVE=docker -
EUREKACLIENTSERVICEURLDEFAULTZONE=http://eureka-server:8761/
eureka - SPRINGCLOUDCONFIGURI=http://config-server:8888 networks: -
microservices-network
```

```
Databases postgres: image: postgres:13 environment: POSTGRESDB:
userdb POSTGRESUSER: postgres POSTGRESPASSWORD: password ports:
- "5432:5432" volumes: - postgresdata:/var/lib/postgresql/data networks: -
microservices-network
```

```
mongodb: image: mongo:4.4 environment:
MONGOINITDBROOTUSERNAME: mongo
MONGOINITDBROOTPASSWORD: password ports: - "27017:27017"
volumes: - mongodb_data:/data/db networks: - microservices-network
```

```
Message Broker kafka: image: confluentinc/cp-kafka:latest environment:
KAFKABROKERID: 1 KAFKAZOOKEEPERCONNECT: zookeeper:2181
KAFKAADVERTISEDLISTENERS: PLAINTEXT://kafka:9092
KAFKAOFFSETSTOPICREPLICATIONFACTOR: 1 depends_on: - zookeeper
networks: - microservices-network
```

```
zookeeper: image: confluentinc/cp-zookeeper:latest environment:
ZOOKEEPERCLIENTPORT: 2181 ZOOKEEPERTICKTIME: 2000 networks: -
microservices-network
```

```
Business Services user-service: build: ./user-service dependson: - postgres
- eureka-server - config-server environment: -
SPRINGPROFILEACTIVE=docker -
SPRINGDATASOURCEURL=jdbc:postgresql://postgres:5432/userdb -
EUREKACLIENTSERVICEURLDEFAULTZONE=http://eureka-server:8761/
eureka - SPRINGCLOUDCONFIGURI=http://config-server:8888 networks: -
microservices-network
```

```
order-service: build: ./order-service dependson: - mongodb - eureka-server -
config-server - kafka environment: - SPRINGPROFILEACTIVE=docker -
SPRINGDATAMONGODBURI=mongodb://
```

```
mongo:password@mongodb:27017/orderdb?authSource=admin -
EUREKACLIENTSERVICEURLDEFAULTZONE=http://eureka-server:8761/
eureka - SPRINGCLOUDCONFIGURI=http://config-server:8888 -
SPRINGKAFKABOOTSTRAPSERVERS=kafka:9092 networks: -
microservices-network
```

```
Monitoring zipkin: image: openzipkin/zipkin ports: - "9411:9411"
networks: - microservices-network
```

```
volumes: postgresdata: mongodbddata:
```

```
networks: microservices-network: driver: bridge ``
```

## Kubernetes Deployment Configuration

```
``yaml
```

# user-service-deployment.yaml

```
apiVersion: apps/v1 kind: Deployment metadata: name: user-service labels:
app: user-service spec: replicas: 3 selector: matchLabels: app: user-service
template: metadata: labels: app: user-service spec: containers: - name: user-
service image: user-service:latest ports: - containerPort: 8080 env: - name:
SPRINGPROFILESACTIVE value: "kubernetes" - name:
SPRINGDATASOURCEURL valueFrom: configMapKeyRef: name: user-
service-config key: database.url - name: SPRINGDATASOURCEUSERNAME
valueFrom: secretKeyRef: name: user-service-secret key: database.username
- name: SPRINGDATASOURCEPASSWORD valueFrom: secretKeyRef: name:
user-service-secret key: database.password resources: requests: memory:
"256Mi" cpu: "200m" limits: memory: "512Mi" cpu: "500m" livenessProbe:
httpGet: path: /actuator/health port: 8080 initialDelaySeconds: 60
periodSeconds: 30 readinessProbe: httpGet: path: /actuator/health port:
8080 initialDelaySeconds: 30 periodSeconds: 10
```

---

```
apiVersion: v1 kind: Service metadata: name: user-service spec: selector:
app: user-service ports: - protocol: TCP port: 80 targetPort: 8080 type:
ClusterIP
```

---

```
apiVersion: v1 kind: ConfigMap metadata: name: user-service-config data:
database.url: "jdbc:postgresql://postgres-service:5432/userdb"
eureka.client.service-url.defaultZone: "http://eureka-service:8761/eureka"
```

---

```
apiVersion: v1 kind: Secret metadata: name: user-service-secret type:
Opaque data: database.username: cG9zdGdyZXM= # postgres (base64
encoded) database.password: cGFzc3dvcmQ= # password (base64 encoded)
``
```

---

## 10. Real-world Case Studies {#case-studies}

### Case Study 1: E-commerce Platform Microservices

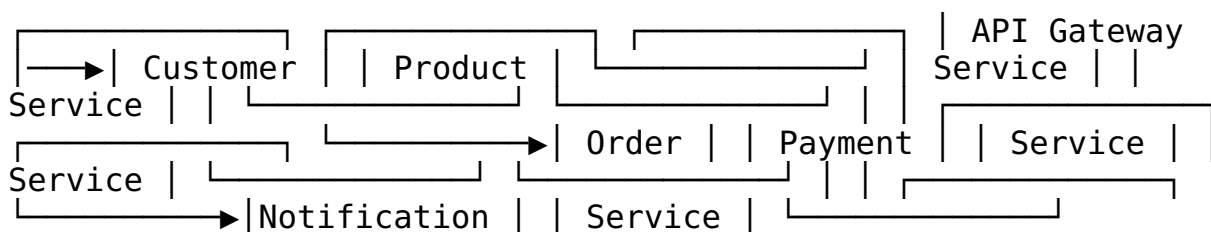
#### Business Requirements:

- Handle high traffic during peak seasons
- Independent scaling of different services
- Fast deployment of new features
- Resilience to service failures
- Support for different teams working independently

#### Architecture Decisions:

- **Domain Decomposition:** Customer, Product, Order, Payment, Notification services
- **Database Strategy:** PostgreSQL for transactional data, MongoDB for catalog, Redis for caching
- **Communication:** Synchronous for critical operations, Async via Kafka for events
- **Security:** OAuth2 with Keycloak for authentication and authorization

#### Service Architecture



#### Order Processing Flow

```
```java @Service @Slf4j public class OrderProcessingService {

@Autowired
private CustomerServiceClient customerClient;

@Autowired
private ProductServiceClient productClient;

@Autowired
private PaymentServiceClient paymentClient;

@Autowired
private KafkaTemplate<String, Object> kafkaTemplate;

@Transactional
public OrderResponse processOrder(OrderRequest request) {
    log.info("Processing order for customer: {}", request.getCustomerId())
```

```

// 1. Validate customer
CustomerDto customer = customerClient.getCustomer(request.getCustomerId());
if (!customer.isEligible()) {
    throw new CustomerNotEligibleException("Customer is not eligible for purchase");
}

// 2. Validate products and check inventory
List<OrderItem> validatedItems = new ArrayList<>();
for (OrderItemRequest itemRequest : request.getItems()) {
    ProductDto product = productClient.getProduct(itemRequest.getProductId());

    if (product.getAvailableQuantity() < itemRequest.getQuantity()) {
        throw new InsufficientInventoryException(
            "Insufficient inventory for product: " + product.getName());
    }

    validatedItems.add(OrderItem.builder()
        .productId(itemRequest.getProductId())
        .quantity(itemRequest.getQuantity())
        .price(product.getPrice())
        .build());
}

// 3. Create order
Order order = Order.builder()
    .customerId(request.getCustomerId())
    .items(validatedItems)
    .totalAmount(calculateTotal(validatedItems))
    .status(OrderStatus.CREATED)
    .orderDate(LocalDateTime.now())
    .build();

Order savedOrder = orderRepository.save(order);

// 4. Send order created event
OrderCreatedEvent event = OrderCreatedEvent.builder()
    .orderId(savedOrder.getId())
    .customerId(savedOrder.getCustomerId())
    .items(savedOrder.getItems())
    .totalAmount(savedOrder.getTotalAmount())
    .build();

kafkaTemplate.send("order-created", event);

// 5. Initiate payment processing
PaymentRequest paymentRequest = PaymentRequest.builder()
    .orderId(savedOrder.getId())
    .customerId(savedOrder.getCustomerId())
    .amount(savedOrder.getTotalAmount())
    .build();

```

```
paymentClient.processPaymentAsync(paymentRequest);

return OrderResponse.builder()
    .orderId(savedOrder.getId())
    .status(savedOrder.getStatus())
    .message("Order created successfully")
    .build();
}

} ````
```

Case Study 2: Banking System Migration

Migration Challenge:

Large banking monolith handling accounts, transactions, loans, and customer management needed to be decomposed for:

- Regulatory compliance isolation
- Independent scaling of transaction processing
- Technology modernization
- Faster feature delivery

Solution Approach:

- **Strangler Fig Pattern:** Gradually extracted services
- **Event Sourcing:** For audit trails and compliance
- **SAGA Pattern:** For distributed transactions
- **Bulkhead Pattern:** Isolate critical operations

Results Achieved:

- **Performance:** 10x improvement in transaction processing
- **Reliability:** 99.9% uptime achievement
- **Deployment:** From monthly to daily deployments
- **Compliance:** Isolated PCI-DSS compliance scope

Key Lessons Learned:

1. Start with clear domain boundaries
2. Invest heavily in monitoring and observability
3. Plan for data consistency challenges
4. Gradual migration reduces risk

11. Security & Data Management {#security}

Security Patterns

- **OAuth2 & JWT:** Token-based authentication

- **API Gateway Security:** Centralized security enforcement
- **Service-to-Service:** Mutual TLS authentication
- **Secrets Management:** External secret stores
- **Network Security:** Private networks and firewalls

Data Management

- **Database per Service:** Data ownership isolation
- **Polyglot Persistence:** Right tool for the job
- **Event-Driven Data:** Eventual consistency patterns
- **Data Synchronization:** CDC and event streaming
- **Backup & Recovery:** Service-specific strategies

Security Implementation Example

OAuth2 Resource Server Configuration

```

```java @Configuration @EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true) public class
SecurityConfig {

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
 return http
 .cors(cors -> cors.configurationSource(corsConfigurationSource()))
 .csrf(csrf -> csrf.disable())
 .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
 .authorizeHttpRequests(authz -> authz
 .requestMatchers("/actuator/health", "/actuator/info").permitAll()
 .requestMatchers("/api/v1/public/**").permitAll()
 .requestMatchers(HttpMethod.POST, "/api/v1/users").hasRole("ADMIN")
 .requestMatchers(HttpMethod.GET, "/api/v1/users/**").hasAnyRole("ADMIN", "USER")
 .anyRequest().authenticated()
)
 .oauth2ResourceServer(oauth2 -> oauth2
 .jwt(jwt -> jwt
 .jwtAuthenticationConverter(jwtAuthenticationConverter())
 .jwtDecoder(jwtDecoder())
)
)
 .build();
}

@Bean
public JwtDecoder jwtDecoder() {
 NimbusJwtDecoder jwtDecoder = JwtDecoders.fromIssuerLocation("http://localhost:8080");

 // Custom validation
 jwtDecoder.setJwtValidator(jwtValidator());

 return jwtDecoder;
}

```

```
}
```

```
@Bean
```

```
public Converter<Jwt, JwtAuthenticationToken> jwtAuthenticationConverter()
 JwtGrantedAuthoritiesConverter authoritiesConverter = new JwtGrantedAu
 authoritiesConverter.setAuthorityPrefix("ROLE_");
 authoritiesConverter.setAuthoritiesClaimName("roles");

 return jwt -> {
 Collection<GrantedAuthority> authorities = authoritiesConverter.co
 return new JwtAuthenticationToken(jwt, authorities);
 };
}
```

```
@Bean
```

```
public Validator<Jwt> jwtValidator() {
 List<Validator<Jwt>> validators = new ArrayList<>();
 validators.add(new JwtTimestampValidator());
 validators.add(new JwtIssuerValidator("http://localhost:8180/realms/mi

 return new DelegatingValidator<>(validators);
}

} ````
```

---

## 12. Conclusion & Next Steps {#conclusion}

### Key Takeaways

1. **Start Simple:** Begin with a modular monolith, evolve to microservices when justified
2. **Domain-Driven Design:** Use DDD to identify natural service boundaries
3. **Spring Cloud Ecosystem:** Leverage proven patterns and tools
4. **Observability First:** Implement comprehensive monitoring from day one
5. **Gradual Migration:** Use Strangler Fig pattern for safe evolution
6. **Testing Strategy:** Implement multi-layer testing approach

### Immediate Next Steps

1. Set up development environment
2. Create proof of concept
3. Implement basic services
4. Add API Gateway and service discovery
5. Implement monitoring and tracing

### Advanced Topics to Explore

- **Service Mesh** (Istio, Linkerd)

- **Event Sourcing & CQRS**
- **Serverless microservices**
- **GraphQL federation**
- **Multi-cloud deployment**

## **Further Learning Resources**

- Spring Cloud documentation
- Microservices patterns books
- Cloud native computing foundation
- Kubernetes certification
- DevOps and SRE practices

## **Success Factors for Spring Boot Microservices**

### **Technical Excellence:**

- Comprehensive automated testing
- Robust CI/CD pipelines
- Comprehensive monitoring and alerting
- Clear API contracts and versioning
- Proper error handling and resilience patterns

### **Organizational Readiness:**

- DevOps culture and practices
- Small, autonomous teams
- Clear service ownership
- Investment in tooling and training
- Commitment to operational excellence

---

## **Thank You!**

### **Ready to build scalable, resilient microservices with Spring Boot**

For questions and further discussion:

- Spring Boot Documentation: <https://spring.io/projects/spring-boot>
- Spring Cloud Documentation: <https://spring.io/projects/spring-cloud>
- Microservices Patterns: <https://microservices.io/>
- Cloud Native Computing Foundation: <https://www.cncf.io/>

This presentation provides comprehensive guidance for designing and implementing microservices architecture using Java Spring Boot, with practical examples, best practices, and real-world insights for Java developers and architects.